DAY 3 · MODULE 3 · 4 HOURS

Agentic AI Foundations for Engineering Teams

Day 2 wired the AWS sandbox + first Lambda. Today we deepen the AI-assisted engineering model: how to prompt for technical work, how to engineer context so Cascade scaffolds against your rules, how to generate + score multiple architectures via Bedrock, and how to spot the failure modes AI-generated code ships when the human review is sloppy.

Accelya Kale · Agentic Development using AWS · Trainer: Jai Singh

Day 2 → Day 3

Yesterday: AWS sandbox verified (Day 1 Bedrock verify succeeded), Day 2 IAM/S3/Lambda/CloudWatch shipped — starter validator Lambda deployed in Lab 3. No Bedrock invocations from D2 Lambda. Today: turn AI assistance from autocomplete into a discipline — prompt + context + multi-architecture + HITL review.

YESTERDAY

Starter Lambda + CloudWatch trace

↓ becomes ↓

TODAY

The artefact you'll critique today (the deliberately weak version) — and the structured prompt that produces a stronger one.

YESTERDAY

Day 1 Bedrock verify (1 invocation)

↓ scales to ↓

TODAY

Bedrock as architecture-options generator: 2-3 variants per prompt, scored against a trainer-provided scorecard.

YESTERDAY

S3 four-zone layout verified

↓ used by ↓

TODAY

Lab 1 writes the prompt; Lab 2 sends it to Bedrock and asks for an event-processing flow that lands in those zones — concrete context, not generic 'AWS Lambda' chat.

YESTERDAY

GitLab repo cloned in Windsurf

↓ hosts ↓

TODAY

/prompts/, /reviews/, /design/ folders — every prompt + review committed as an audit artefact (set up today, used D4 onward).

YESTERDAY

IAM least-privilege intro

↓ appears in ↓

TODAY

5-dim review checklist: Security is one of the five gates every AI output passes through before merge.

YESTERDAY

1 Bedrock invocation used so far (Day 1 verify)

↓ +1-4 today ↓

TODAY

Lab 2 = 1 required Bedrock invocation (up to 3-4 if regenerated). Lab 3 Cascade usage is IDE-side, not Bedrock. ~43-46 left for D4-D12.

Why an AI-foundations day before any pipeline code

THE TRAP — 'CASCADE WROTE IT, SHIP IT'

Without prompt + context discipline, AI-generated code ships hallucinated APIs, missing error paths, plausible-but-wrong field names, and zero observability. Cascade can scaffold a 200-line Lambda in 90 seconds — and it'll look right at first glance.

By Day 12 you have 10 Lambdas + 1 state machine. If today's review discipline is sloppy, every one of them carries a defect that's ~10× harder to find later than to prevent now.

THE DISCIPLINE — PROMPT + CONTEXT + REVIEW

Three habits make AI assistance reliable for engineering:

1. PROMPT the task in 5 named sections (role, context, task, output format, review notes).
2. CONTEXT-engineer the project so Cascade should pick up on every invocation (Lab 1 verifies) (.windsurfrules + folder layout + payload schemas + business glossary).
3. REVIEW every output against the 5-dim HITL checklist (correctness, security, observability, scalability, cost).

Today you build all three — using airline data, Bedrock, and Cascade — so D4-D12 inherits the discipline.

BY THE END OF TODAY YOU WILL BE ABLE TO

By the end of Day 3 you can…

PROMPT

Author a 5-section technical prompt (role · context · task · output format · review notes) for an airline event-processing flow.

CONTEXT

Set up /prompts/ /reviews/ /design/ folders + a v1 .windsurfrules with REPO RULES + CONSTRAINTS + EXAMPLES + REVIEW that Cascade should pick up on every prompt from D4 onward (Lab 1 verifies).

GENERATE

Ask Bedrock (Converse API) for 2-3 architecture variants for a single business requirement; receive structured JSON back; compare on a trainer-provided 5-dim scorecard.

SPOT

Identify the 6 most common failure modes in AI-generated engineering code: hallucinated APIs, plausible-but-wrong fields, missing error paths, over-abstraction, silent swallow, IAM wildcards.

REVIEW

Apply the 5-dim HITL checklist (correctness, security, observability, scalability, cost) to a deliberately weak Cascade-generated function; write the review as HITL_REVIEW.md and commit it.

REFINE

Iterate the winning prompt based on the review and re-run; show the second-iteration output is materially better than the first.

AGENDA · 4 HOURS

Four parts. Concepts → Design → Labs → Wrap.

01 Concept briefing

Six concepts. What 'agentic for engineering' means in 2026 vs autocomplete. Anatomy of a 5-section technical prompt. Context engineering and what Cascade actually reads. Multi-architecture generation pattern. Six AI failure modes. The 5-dim HITL review checklist as a working artefact.

TIME

~30-45 min

02 Guided design

Three guided design slides. The repo layout for the next 9 days. The .windsurfrules v1 template (four sections — REPO RULES, CONSTRAINTS, EXAMPLES, REVIEW). The trainer-provided architecture scorecard (5 rows × 5 columns) you'll fill out in Lab 2.

TIME

~45-60 min

03 Three labs

Lab 1 — author the 5-section prompt for a cancellation-event flow + commit (45 min). Lab 2 — Bedrock generates 2-3 architecture variants; pair-score on the 5-dim scorecard; commit decision MR (45 min). Lab 3 — Cascade-critique a deliberately weak function; refine the prompt; re-run; show the diff (45 min).

TIME

~135 min

04 Review & wrap

Five Day-3 anti-patterns to avoid. End-of-module cleanup (light — no AWS provisioning). Day-3 in numbers. Bridge to Day 4 — Batch Pipeline Design on AWS, where today's prompt + .windsurfrules drive the design doc.

TIME

~20-30 min

01

PART 1 · ~30-45 MINUTES

Concept briefing

Six concepts that turn AI from autocomplete into engineering discipline. Agentic vs autocomplete. The 5-section prompt anatomy. Context engineering. Multi-architecture generation. Six failure modes in AI-generated code. The 5-dim HITL review checklist.

Three modes of AI-assisted engineering. This programme lives in the middle one.

Most teams adopt AI as autocomplete (Tab key) and stop there. The middle mode — agentic engineering — is where the productivity unlocks live, and where the review discipline matters most.

Three modes — autonomy spectrum vs review depth

MODE 1 · AUTOCOMPLETE
Tools: Cascade Tab completions, IDE inline suggestions
Autonomy: LOW (single line / single function fragment)
Review: glance + accept; ~1 second per suggestion
Risk: LOW per suggestion; HIGH cumulatively (silent style drift)
Right tool when: writing boilerplate, type signatures, repeated patterns

MODE 2 · AGENTIC ENGINEERING (this programme)
Tools: Cascade in Windsurf — multi-file edits, plans, terminal awareness, .windsurfrules
Autonomy: MEDIUM (whole-feature scaffold; multiple files; runs tests)
Review: 5-dim HITL checklist on every output; ~5-10 min per artefact
Risk: MEDIUM if reviewed; HIGH if treated like autocomplete
Right tool when: scaffold a Lambda + tests + IaC + review the diff before merge

MODE 3 · FULL AUTONOMY (NOT this programme)
Tools: Background agents, autonomous CI bots, agentic-CI tools
Autonomy: HIGH (opens MRs unattended; deploys via approval gates)
Review: asynchronous PR review + smoke-test gating
Risk: HIGHEST without robust gates; needs all of: SAM IaC + scanners + alarms + rollback
Right tool when: mature platform team with D11-grade CI/CD; this course teaches the FOUNDATION not the autonomy

01 **Autocomplete = 1 line**

Tab completions. Glance + accept. Cumulative drift is the risk.

02 **Agentic = 1 feature**

Multi-file scaffold + tests + diff. 5-dim review per output. THIS programme.

03 **Full autonomy = 1 MR**

Background agents open MRs. Needs D11-grade gating to be safe. NOT taught here.

04 **Discipline scales with autonomy**

More autonomy → more rigorous review. The middle mode is the highest leverage.

05 **5-dim HITL is the gate**

Concept 6 + every D4-D12 module. Same 5 dimensions; every artefact passes through them.

ROLE · CONTEXT · TASK · OUTPUT FORMAT · REVIEW NOTES. Order matters. So does the airline vocabulary.

Generic 'write me a Lambda that processes orders' produces generic output. A 5-section prompt with airline vocabulary + explicit constraints produces something you can ship — or at least review meaningfully.

The 5 sections — what each one is for + a worked example

```
## SECTION 1 — ROLE
## Tells the model what kind of expert it is. Pins vocabulary + assumptions.
  You are a senior AWS data engineer specialising in airline real-time event
  pipelines. You design for partial failure, idempotency, and cost-aware sampling.
  You write Python 3.12 + boto3; you favour explicit error classification over
  silent swallow.

## SECTION 2 — CONTEXT (the project + the data)
## Business setting + airline vocabulary + payload shape + constraints.
  Accelya processes airline cancellation events arriving via Kinesis Data Streams.
  Each event is a JSON envelope with: event_id (UUID), event_type ('cancellation'),
  airline_code (IATA 2-letter), pnr, booking_date, cancel_reason_code, fare_amount.
  Required fields: event_id, event_type, airline_code, pnr, booking_date.
  Sandbox caps: AWS Lambda 10 functions / 2048 MB / 30s timeout; AWS Bedrock
  50 invocations + 20K tokens per learner per session.

## SECTION 3 — TASK (what to produce; unambiguous)
  Propose 2-3 architecture options for ingesting + validating + sampled-enriching
  these cancellation events on AWS, landing valid records in s3://.../enriched/
  and invalid in s3://.../quarantine/. Each option must be cost-aware and respect
  the sandbox caps above.

## SECTION 4 — OUTPUT FORMAT (machine + human readable)
  Return JSON only, conforming to this schema:
    // JSON-style schema below; actual output must be VALID JSON (double quotes; no comments).
    { "options": [
        { "name": "string", "services": ["string"], "flow": "string", "pros": ["string"], "cons":
["string"],
          "cost_drivers": ["string"], "relative_cost": "low|medium|high", "sandbox_compatible":
true } ] }
    // relative_cost / cost_drivers are illustrative only — actual pricing taught in D10.
  No prose preamble; no markdown fences.

## SECTION 5 — REVIEW NOTES (constraints + non-goals)
```

01 ROLE first

Pins vocabulary + defaults. 'Senior airline data engineer' beats 'helpful AI'.

02 CONTEXT = data + caps

Payload shape + airline glossary + sandbox caps. Without this Bedrock hallucinates fields.

03 TASK = unambiguous

'2-3 options for ingest + validate + sampled-enrich' beats 'design the pipeline'.

04 OUTPUT FORMAT = JSON schema

Lets you machine-parse the response + score it against a scorecard. No markdown fences.

05 REVIEW NOTES = non-goals

What NOT to use. Closes off the most common AI mistakes (out-of-scope services, banned features).

.windsurfrules + folder layout + airline glossary + payload examples = the project's running context.

A good prompt produces good output once. A good context produces good output every time. Cascade should pick up .windsurfrules on every prompt — Lab 1 verifies this with a fresh chat. That's where you encode the rules of the project so you don't have to repeat them.

.windsurfrules v1 — four sections (built in Lab 1; extended D4 onward)

```
## .windsurfrules — committed to repo root; Cascade should pick up on every prompt (Lab 1
verifies)

## REPO RULES
  - All Lambdas: src/<component>/handler.py + src/<component>/test_handler.py
  - Shared types: src/common/exceptions.py (PipelineError ladder)
  - IaC: infra/template.yaml (SAM)
  - Prompts:  prompts/v<N>_<purpose>.txt (committed, audit-able)
  - Reviews:  reviews/HITL_REVIEW-<artefact>.md (5-dim score per output)
  - Synthetic data only: no real customer / production data anywhere
  - Airline vocabulary mandatory (PNR, airline_code, fare_class, OD, settlement)
    NOT generic ('order', 'user', 'item' are forbidden in payload examples)

## CONSTRAINTS
  - Region:    All AWS API calls originate from ap-south-1 (Mumbai).
               Bedrock uses APAC cross-region inference profile;
               AWS may route within supported APAC regions.
  - Lambda:    max 10 functions per learner; 2048 MB per function;
               default timeout 30s (modules may override per-lab — e.g. SageMaker wrapper, ETL);
               Event-driven Lambda only (no always-on worker pattern; sandbox)
  - Bedrock:   max 50 invocations + 20,000 tokens per learner per session;
               BEDROCK_MODEL_ID = 'apac.anthropic.claude-3-5-sonnet-20241022-v2:0'
               (full APAC cross-region inference profile ID; reference via env var);
               NO Bedrock Studio, Knowledge Bases, Agents, AgentCore
  - SageMaker: max 1 endpoint; for provisioned endpoints use allowed CPU instance families only;
NO GPUs (Day 9 uses Serverless: MemorySizeInMB + MaxConcurrency, not instance type)
  - Glue:      max 3 jobs / 4 DPU total / 6 DPU-h cumulative; sessions disabled
  - FIS:       max 20 minutes action time per lab
  - GitLab:    Free tier (no merged-results pipelines; manual cleanup-sandbox)
  - Cost Explorer API: NOT enabled in sandbox (use CW metric proxies instead)
  - AWS Budgets: NOT supported (use CW metric-math alarms)

## EXAMPLES
```

01 — Picked up on every prompt

Cascade should pick up .windsurfrules; Lab 1 verifies in a fresh chat. You don't paste it — but you do maintain it.

02 — REPO RULES = layout

Where files live + naming conventions + the audit folders (prompts/ reviews/).

03 — CONSTRAINTS = sandbox caps

Every cap from the 322-row XLSX + the Bedrock/SageMaker/FIS/GitLab/CE realities.

04 — EXAMPLES = airline payloads

PNR, airline_code, BSP cycle, settlement_id. Cascade pattern-matches; no generics.

05 — REVIEW = 5-dim gate

Concept 6 + Lab 3. Every output passes through the same checklist before merge.

Ask Bedrock for 2-3 options with pros/cons. Score on a 5-dim scorecard. Pick the winner with reasons.

A single 'best architecture' answer hides the trade-offs. Asking for 2-3 options forces the trade-offs into the open — and the scoring exercise teaches the team how to weigh them. This pattern repeats every module D4-D12: Cascade proposes; humans choose with reasons.

The Converse pattern — one prompt, multiple options, structured output

```python
# scripts/generate_architectures.py — Lab 2 deliverable
import json, os, re, boto3
client = boto3.client('bedrock-runtime', region_name='ap-south-1')
MODEL_ID = os.environ.get('BEDROCK_MODEL_ID',
    'apac.anthropic.claude-3-5-sonnet-20241022-v2:0')

def strip_fences(s: str) -> str:
    # Robust JSON-fence cleanup: trims whitespace, strips ``` or ```json fences
    # at start AND end, then falls back to first '{' .. last '}' if Bedrock
    # leaked prose around the JSON body.
    s = s.strip()
    s = re.sub(r'^```[a-zA-Z]*\n?', '', s)
    s = re.sub(r'\n?```\s*$', '', s).strip()
    if not s.startswith('{'):
        first, last = s.find('{'), s.rfind('}')
        if first >= 0 and last > first:
            s = s[first:last+1]
    return s.strip()

with open('prompts/v1_arch_options.txt') as f:
    prompt = f.read()

response = client.converse(
    modelId=MODEL_ID,
    system=[{'text': 'Output JSON only; no prose; no markdown fences.'}],
    messages=[{'role': 'user', 'content': [{'text': prompt}]}],
    inferenceConfig={'maxTokens': 2000, 'temperature': 0.2},
)
raw = response['output']['message']['content'][0]['text']
parsed = json.loads(strip_fences(raw))
assert 'options' in parsed, f"expected top-level 'options' key; got {list(parsed.keys())}"
options = parsed['options']
assert 2 <= len(options) <= 3, f"expected 2-3 options, got {len(options)}"
```

01 — Converse, not InvokeModel

Modern surface; consistent across model families; structured response shape.

02 — system + user split

system = 'JSON only'; user = the 5-section prompt. Keeps formatting separate from intent.

03 — temperature 0.2

Low for architecture options (we want consistent JSON); higher for creative tasks.

04 — Validate the response

Assert option count (2-3); FLAG (don't crash) if sandbox_compatible=False. Scorecard disqualifies.

05 — Score, don't argue

Use the trainer-provided scorecard. Pair-pick the winner. Decision MR with reasons.

Hallucinated APIs · Plausible-but-wrong fields · Missing error paths · Over-abstraction · Silent swallow · IAM wildcards.

These are the six things AI gets confidently wrong. The 5-dim review checklist catches all of them — but only if reviewers know what they're looking for. Today you see each failure mode in concrete form (Lab 3 weak function), so D4-D12 reviews can name them quickly.

Six failure modes — what each looks like in airline pipeline code

```
1. HALLUCINATED APIS
   Symptom:   boto3.client('s3').get_object_async() — function doesn't exist
   Symptom:   kinesis.put_records_batch() — wrong name; real API is put_records()
   Catch via: actually run the unit test; check boto3 docs for the method

2. PLAUSIBLE-BUT-WRONG FIELD NAMES
   Symptom:   record['booking_id'] — but the envelope uses 'pnr' (airline-specific)
   Symptom:   event['airline'] — but spec says event['airline_code'] (IATA 2-letter)
   Catch via: pin the schema in Concept 3's EXAMPLES; review against actual payload

3. MISSING ERROR PATHS
   Symptom:   try: bedrock.invoke_model(...) except Exception: pass  (swallows everything)
   Symptom:   no handling for ProvisionedThroughputExceededException on Kinesis
   Catch via: 5-dim 'Correctness' + 'Observability' dims explicitly check error classification

4. OVER-ABSTRACTION
   Symptom:   AbstractEventProcessorFactoryStrategy + 200 lines for a 30-line problem
   Symptom:   generic 'Pipeline' / 'Handler' base classes with single subclass
   Catch via: 5-dim 'Scalability' includes 'maintainable by another engineer in 6 months'

5. SILENT SWALLOW (the highest-cost failure mode)
   Symptom:   except KeyError: return None  (downstream sees None; never knows why)
   Symptom:   logger.debug(e) instead of logger.error / classifier raise
   Catch via: 5-dim 'Observability' — every error must classify (transient/data/config) + log

6. IAM WILDCARDS
   Symptom:   {'Action': 's3:*', 'Resource': '*'} in the IAM policy Cascade scaffolds
   Symptom:   missing Conditions (no s3:prefix, no aws:SecureTransport)
```

01 **Hallucinated APIs**

boto3 method that doesn't exist. Run the test.

02 **Plausible-wrong fields**

'booking_id' instead of 'pnr'. Pin the schema in EXAMPLES.

03 **Missing error paths**

No catch for ProvisionedThroughputExceededException. Correctness + Observability dim.

04 **Over-abstraction + silent swallow**

AbstractFactoryStrategy + except: pass. Scalability + Observability dim.

05 **IAM wildcards**

s3:* on Resource:*. Security dim + D10 tightening pass.

Correctness · Security · Observability · Scalability · Cost. Score 0-3 each. Pass: >=12/15 AND every dim >=2/3.

The 5-dim checklist is the spine of every D4-D12 module. It is NOT a bureaucratic form — it's the contract that says 'this AI-generated artefact is reviewed enough to merge'. Today you fill one out for the first time (Lab 3); tomorrow it scales to every Cascade output across the programme.

5 dimensions × 0-3 each = max 15 — what each dim asks

DIM 1 · CORRECTNESS
Does the code actually do what the prompt asked?
Real APIs (no hallucinations)? Edge cases handled (empty input, malformed payload)?
3 = matches spec + edge cases tested
2 = matches spec; happy path only
1 = matches spec but with a missing field or wrong API name
0 = doesn't compile / hallucinated APIs

DIM 2 · SECURITY
IAM least-privilege (no wildcards)? Secrets out of code (env vars / Secrets Manager)?
PII handling explicit? Injection-safe (no eval, no shell concat)?
3 = ARN-scoped IAM + named verbs + Conditions; secrets via env; PII tagged
2 = no wildcards; secrets via env; PII not addressed
1 = some wildcards present
0 = Action: '*' / Resource: '*' / hardcoded secrets

DIM 3 · OBSERVABILITY
Structured JSON logs? correlation_id propagated? Errors classified (transient/data/config)?
EMF metrics emitted (records_in / records_failed / etc.)?
3 = powertools Logger + correlation_id + EMF + classified errors
2 = JSON logs + classified errors; no metrics yet
1 = JSON logs only
0 = print statements / silent except: pass

DIM 4 · SCALABILITY
Will it survive 10× the data? Idempotent? Backpressure-aware? Maintainable in 6 months?
3 = idempotent + bounded memory + maintainable + tested at 10× scale
2 = idempotent + bounded; maintainability OK

01 **5 dims × 0-3 = max 15**

Pass: >=12/15 + every dim >=2/3. Lower = revise.

02 **Correctness = real APIs + edges**

Hallucinated method = 0. Happy path only = 2. Edge cases tested = 3.

03 **Security = no wildcards + secrets**

ARN-scoped + Conditions + env-var secrets = 3. Action:* = 0.

04 **Observability = JSON + classified**

powertools + correlation_id + EMF = 3. except: pass = 0.

05 **Scalability + Cost**

Idempotent + bounded + maintainable. Right service. Sampled where applicable.

PART 1 WRAP-UP

AI-foundations vocabulary set

Three modes (autocomplete · agentic · full autonomy). 5-section prompt anatomy. .windsurfrules + folder layout = context engineering. Multi-architecture via Bedrock Converse. Six AI-code failure modes. 5-dim HITL review checklist. Now: guided design — the repo layout you'll use for D4-D12 + the v1 .windsurfrules + the trainer scorecard.

VOCABULARY

Agentic engineering, 5-section prompt, .windsurfrules, Converse API, multi-architecture options, 5-dim HITL, six failure modes.

WHAT'S NEW

Today is the FIRST module to commit /prompts/, /reviews/, /design/ folders + .windsurfrules v1. Every D4-D12 module extends the .windsurfrules with a new context layer (BATCH, STREAM, MODEL, OBSERVABILITY, DEPLOY).

WHAT'S NEXT

Three guided-design slides: repo + audit folders, .windsurfrules v1 template, trainer-provided architecture scorecard.

02

PART 2 · ~45-60 MINUTES

Guided design

Three guided-design slides. The repo + audit folders that travel through D4-D12. The .windsurfrules v1 template (4 sections — REPO RULES, CONSTRAINTS, EXAMPLES, REVIEW). The trainer-provided architecture scorecard (5 rows × 5 columns) for Lab 2 + future modules.

Every artefact this programme produces lives in one of five top-level folders. AI prompts are first-class citizens — they're committed alongside code so reviewers can audit the prompt that produced any output.

THE LAYOUT

Five top-level folders. Each has a single, named purpose.

```
accelya-airline-pipeline/          # Your GitLab repo root
├── prompts/                       # ALL prompts (committed audit trail)
│   └── v<N>_<purpose>.txt         # versioned: v1_arch_options.txt, v8_batch.txt, ...
├── reviews/                       # 5-dim HITL_REVIEW.md per artefact
│   └── HITL_REVIEW-<artefact>.md  # one per Cascade output that was reviewed
├── design/                        # Design docs (D4 onward) before code lands
│   └── m<N>-<topic>.md            # m4-batch-pipeline-design.md, etc.
├── src/                           # Code (D4 onward)
│   ├── common/exceptions.py       # PipelineError ladder (D4 introduces)
│   └── <component>/handler.py     # one per Lambda
├── infra/                         # SAM template + per-resource snippets (D11 consolidates)
└── .windsurfrules                 # Cascade should pick up on EVERY prompt (Lab 1 verifies)
```

WHY PROMPTS ARE COMMITTED

If you can't reproduce the prompt, you can't review the artefact. Audit + reproducibility.

```
## A real D4 review looks like this:
## Reviewer reads: src/validator/handler.py
## Reviewer reads: prompts/v8_batch.txt   ← the prompt that produced it
## Reviewer reads: reviews/HITL_REVIEW-validator.md   ← the 5-dim score
##
## Without the committed prompt, the reviewer can't tell whether:
##   (a) the code is correct but the prompt was vague
##   (b) the prompt was tight but Cascade hallucinated
##   (c) someone hand-edited Cascade's output without re-reviewing
## Committed prompts close all three holes.
```

LAB 1 SETS UP THE FOLDERS

Today's labs commit the empty folders + the first prompt + .windsurfrules v1. D4 onward fills them.

```
git checkout -b module-3-$LEARNER
mkdir -p prompts reviews design
touch prompts/.gitkeep reviews/.gitkeep design/.gitkeep
git add prompts/ reviews/ design/
git commit -m "module-3: scaffold prompts/ reviews/ design/ audit folders"
git push -u origin module-3-$LEARNER
```

The v1 .windsurfrules covers everything Cascade needs to know on Day 3 + Day 4. Each later module extends it with a new context layer: D4 adds BATCH CONTEXT (the 8th layer), D7 adds STREAM CONTEXT, D8 STATE CONTEXT, D9 MODEL CONTEXT, D10 OBSERVABILITY CONTEXT, D11 DEPLOY CONTEXT.

TEMPLATE — paste into .windsurfrules at repo root

Same content as Concept 3 + REPO RULES section. Cascade should pick up on every prompt (Lab 1 verifies).

```
## REPO RULES
  - Lambdas: src/<component>/handler.py +
src/<component>/test_handler.py
  - Shared types: src/common/exceptions.py
(PipelineError ladder)
  - IaC:      infra/template.yaml (SAM)
  - Prompts:  prompts/v<N>_<purpose>.txt
(committed)
  - Reviews:  reviews/HITL_REVIEW-<artefact>.md
(5-dim per output)
  - Synthetic data only; airline vocab
mandatory (PNR, fare_class, OD, settlement)

## CONSTRAINTS
  - Region:    All AWS API calls originate from
ap-south-1.
               Bedrock uses APAC cross-region
inference profile (may route within APAC).
  - Lambda:    max 10 / 2048 MB / default 30s
(modules may override per-lab)
  - Bedrock:   max 50 inv + 20K tok;
               BEDROCK_MODEL_ID =
'apac.anthropic.claude-3-5-sonnet-20241022-
v2:0';
               NO Studio / KB / Agents /
AgentCore
  - SageMaker: max 1 endpoint; provisioned:
allowed CPU families only; Day 9 uses
Serverless (MemorySizeInMB + MaxConcurrency);
NO GPUs
  [illegible]
sessions disabled
```

HOW LATER MODULES EXTEND IT

D4-D11 each add ONE context layer in a fixed slot. v1 → v9 across the programme.

```
v1 (TODAY)       4 sections — REPO RULES +
CONSTRAINTS + EXAMPLES + REVIEW
v2 (D4)          + 8th BATCH CONTEXT layer
(Records[], partial-batch protocol,
idempotency)
v3 (D5+D6)       + 9th CATALOG/ANALYTICS
CONTEXT (Glue table, partition keys, workgroup)
v4 (D7)          + 10th STREAM CONTEXT (stream
name, shard count, PK strategy)
v5 (D8)          + 11th STATE CONTEXT for
streams (status-model idempotency, sampling)
v6 (D9)          + 12th MODEL CONTEXT (which
model, input shape, confidence threshold)
v7 (D10)         + 13th OBSERVABILITY CONTEXT
(correlation_id, EMF, trace propagation)
v8 (D11)         + 14th DEPLOY CONTEXT (stack
name, CI rules, scanner config, rollback)
v9 (D12)         capstone reuses v8 — no new
layer; review-only day
```

TEST IT IN WINDSURF

Lab 1 verifies Cascade actually picks up .windsurfrules on a hello-world prompt.

```
# After committing .windsurfrules to repo root,
open a NEW Cascade chat:
@cascade  Scaffold a hello-world Lambda for the
cancellation envelope.

# Cascade output should:
#   - Use src/<component>/handler.py path (REPO
RULES)
#   - Reference the airline cancellation
envelope fields (EXAMPLES)
#   - Stay under 30s timeout / 2048 MB
(CONSTRAINTS)
#   - NOT propose Bedrock Studio / Knowledge
Bases (CONSTRAINTS)
#   - Include structured logger (REVIEW)
# If any of those fail, .windsurfrules is not
being read; revise + re-run.
```

Lab 2 generates 2-3 architecture options via Bedrock. The team scores each option on this scorecard. Decision MR captures the winner + rationale + the runner-up's strongest dim (so the team remembers what they gave up).

THE SCORECARD — saved to /reviews/m3-arch-scorecard.csv

Cell values 0-3. Bottom row sums per option. Pass-to-shortlist: >=10/15.

```
Dimension          | Option A | Option B |
Option C
-------------------|----------|----------|-----
-----
Correctness  (0-3) |    _     |    _     |    _
Security     (0-3) |    _     |    _     |    _
Observability(0-3) |    _     |    _     |    _
Scalability  (0-3) |    _     |    _     |    _
Cost         (0-3) |    _     |    _     |    _
-------------------|----------|----------|-----
-----
TOTAL (max 15)     |   __     |   __     |   __
Sandbox-compatible?|  Y/N     |  Y/N     |  Y/N

Decision rule:
  - Sandbox-compatible = N → instant
disqualify, regardless of total
  - Highest total wins; tie-break on Security
(reflects programme priorities)
  - <10/15 across the board → revise prompt +
re-run (no shortlist)
```

WORKED EXAMPLE — cancellation event flow

Three plausible options for a real prompt; what scoring looks like.

```
Option A — S3 event-driven (S3 PUT → Lambda
validate → S3 enriched/quarantine)
  Correctness 3  Security 2  Observability 2
Scalability 2  Cost 3   = 12/15 ✓ shortlist
  Sandbox-compatible: Y

Option B — Kinesis stream (Kinesis → Lambda
consumer → DynamoDB idempotency → enriched/)
  Correctness 3  Security 3  Observability 3
Scalability 3  Cost 2   = 14/15 ✓ shortlist
  Sandbox-compatible: Y

Option C — SQS-fanout (SNS → SQS per consumer →
multiple Lambdas)
  Correctness 2  Security 2  Observability 1
Scalability 3  Cost 1   = 9/15 ✗ no shortlist
  Sandbox-compatible: Y

Decision: Option B wins (14 > 12). Decision MR
notes: chose B over A for stronger
Observability + Scalability; gave up some Cost
(Kinesis shard hour) — acceptable for
real-time SLA. Option C scoring is in the MR
for the audit trail.
```

WHY THIS PATTERN REPEATS

Same 5 dims as the HITL code review. Same 0-3 scoring. Different pass bars by purpose. Reusable D4-D12.

```
## ARCHITECTURE-OPTIONS scorecard (Lab 2 today;
D4 reuses it; D7 reuses it again):
##   Same 5 dims; same 0-3 scoring.
##   Shortlist bar: >=10/15 AND
sandbox_compatible=True (looser; design-stage
choice).
## CODE-LEVEL HITL_REVIEW (Lab 3 today; every
D4-D12 lab):
##   Same 5 dims; same 0-3 scoring.
##   Pass bar: >=12/15 AND every dim >=2/3
(tighter; per artefact).
## D11 9th HITL dim (Deploy Auditability):
##   Adds a 6th column for IaC/CI artefacts.
Same 0-3.
## D12 capstone 10-dim rubric:
##   Different shape (10 dims instead of 5;
pass at 22/30 instead of 12/15) — but
##   the *idea* of an explicit dim-scored
review is the same.
##
## Today is where this pattern is INTRODUCED.
After today, every artefact in the
## programme is reviewed against an explicit
dim-scored checklist before merge.
```

PART 2 WRAP-UP

Repo + .windsurfrules + scorecard ratified

Five top-level folders. v1 .windsurfrules with 4 sections. Trainer-provided architecture scorecard. All three artefacts committed in Lab 1; the scorecard gets used in Lab 2; the HITL review template gets used in Lab 3.

REPO LAYOUT

/prompts/ /reviews/ /design/ /src/ /infra/ — set up Day 3, used D4-D12. Prompts committed alongside code.

.WINDSURFRULES v1

4 sections (REPO RULES + CONSTRAINTS + EXAMPLES + REVIEW). Cascade should pick up on every prompt (Lab 1 verifies). Extends D4 onward.

SCORECARD

5 dims × 0-3 = max 15. Pass to shortlist >=10/15. Same shape as the HITL code review.

03

PART 3 · ~135 MINUTES

Three labs

Lab 1 — author the 5-section prompt + commit /prompts/, /reviews/, /design/ + .windsurfrules v1 (45 min). Lab 2 — Bedrock generates 2-3 architecture variants; pair-score on scorecard; commit decision MR (45 min). Lab 3 — Cascade-critique a deliberately weak function; refine the prompt; re-run; show the diff (45 min).

LAB 1 · ~45 MINUTES

Author 5-section prompt + scaffold /prompts/ /reviews/ /design/ + .windsurfrules v1

BRIEF

Pair-work. Open Windsurf in your cloned repo. Create the three audit folders + .windsurfrules v1 + prompts/v1_arch_options.txt — the 5-section prompt for Lab 2's architecture-options exercise. Verify Cascade picks up .windsurfrules by asking it to scaffold a hello-world Lambda + checking the output uses src/<component>/handler.py + airline envelope fields (NOT generic 'order' fields). Commit + push module-3-<learner> branch.

TIME

45 min

LEVEL

Working+

DELIVERABLE

prompts/v1_arch_options.txt (5-section prompt for cancellation event-processing flow) + .windsurfrules v1 (4 sections per Guided Design slide 2) + /prompts/, /reviews/, /design/ folders committed (with .gitkeep) + /reviews/m3-cascade-rules-test.md (test-pass evidence) + MR open with pair partner tagged

Six steps. Branch → folders → .windsurfrules → prompt → Cascade test → MR.

1 Branch + scaffold audit folders

```
export LEARNER=jai          # use your name
git checkout main && git pull --ff-only
git checkout -b module-3-$LEARNER
```

EXPECTED

On module-3-<learner>. /prompts/ /reviews/ /design/ all present (with .gitkeep).

2 Author .windsurfrules v1

```
# Paste the SHORT lab template (compact form of Guided Design slide 9):
# This includes every sandbox cap learners need today; full version with Glue/FIS
# caps lives in the Concept-3 slide and is what gets committed for D5+ work.
```

EXPECTED

.windsurfrules at repo root with 4 sections (paste from Guided Design slide 2).

3 Author the 5-section prompt for Lab 2

```
# Use Concept 2's worked example as your starting template; customise SECTION 2's
# CONTEXT to your specific synthetic dataset (use the cancellation envelope fields).
```

EXPECTED

prompts/v1_arch_options.txt with all 5 sections + airline cancellation context.

4 Test Cascade picks up .windsurfrules

```
# Open a NEW Windsurf chat (so context is fresh):
@cascade  Scaffold a hello-world Lambda for the cancellation envelope.
```

EXPECTED

Cascade output uses src/<component>/handler.py path + airline envelope fields (no generic 'order').

5 Pair-review the prompt + .windsurfrules

```
# Pair partner reads:
#   .windsurfrules
#   prompts/v1_arch_options.txt
```

EXPECTED

Pair partner reads both; suggests one improvement to the prompt; you accept or reject with reason.

6 Commit + push + open MR

```
git status   # confirm: .windsurfrules, prompts/v1_arch_options.txt, .gitkeep files
git commit -m "module-3: scaffold prompts/+reviews/+design/ folders + .windsurfrules v1 +
v1_arch_options.txt"
```

EXPECTED

MR open on GitLab; pair partner tagged; 1 commit (or 2 if pair-review change accepted).

Success criteria & common gotchas

✓ YOU ARE READY IF…

- .windsurfrules at repo root (4 sections present: REPO RULES + CONSTRAINTS + EXAMPLES + REVIEW)
- prompts/v1_arch_options.txt has all 5 sections + airline cancellation envelope in CONTEXT
- Cascade test transcript shows .windsurfrules being respected (src/ path + airline fields)
- /prompts/, /reviews/, /design/ folders committed with .gitkeep
- MR open; pair partner tagged

⚠ TOP GOTCHAS

Generic vocabulary in EXAMPLES

Wrote 'order' / 'user' / 'item' instead of 'PNR' / 'fare_class' / 'settlement'. Cascade pattern-matches; generics produce generic output. Fix the EXAMPLES block.

CONSTRAINTS block missing the new caps

Cost Explorer NOT enabled + AWS Budgets unsupported + GitLab Free tier — all three came from client confirmations. Make sure they're in v1 so Cascade doesn't propose CE/Budgets in Lab 2.

5-section prompt missing OUTPUT FORMAT

Without the JSON schema in SECTION 4, Lab 2 can't machine-parse Bedrock's response. Always include the schema.

Cascade ignored .windsurfrules

If the test prompt produced generic 'orders' fields, .windsurfrules is at the wrong path or the chat was opened before the file was saved. Move the file to repo root + open a NEW Cascade chat.

REVIEW NOTES too vague

'Be careful with cost' is not a review note. List specific bans: NO SageMaker / NO Bedrock Agents / NO EventBridge Pipes.

LAB 2 · ~45 MINUTES

Bedrock generates 2-3 architecture variants + pair-score on the 5-dim scorecard

BRIEF

Pair-work. Run prompts/v1_arch_options.txt against Bedrock via the Converse API (scripts/generate_architectures.py per Concept 4). Capture the JSON response to outputs/m3-arch-options.json. Pair-score each option on the 5-dim scorecard (/reviews/m3-arch-scorecard.csv). Pick the winner; write decision MR with rationale + what you gave up vs the runner-up. 1 required Bedrock invocation; up to 3-4 if you regenerated.

TIME

45 min

LEVEL

Working+

DELIVERABLE

scripts/generate_architectures.py (Cascade-authored against Concept 4 template) + outputs/m3-arch-options.json (Bedrock's JSON response) + /reviews/m3-arch-scorecard.csv (5 dims × N options; totals + sandbox-compatible flag) + /reviews/m3-arch-decision-MR.md (winner + rationale + runner-up's strongest dim) + MR shows 2 commits

Six steps. Cascade-author script → run → JSON parse → pair-score → decide → commit.

1 Cascade-author the Converse script

```
# In Windsurf chat (loads .windsurfrules automatically):
@cascade  Create scripts/generate_architectures.py per Concept 4 of Day 3.
          It should: read prompts/v1_arch_options.txt; call Bedrock Converse with
```

EXPECTED

scripts/generate_architectures.py per Concept 4 template; reads prompts/v1_arch_options.txt; writes outputs/m3-arch-options.json.

2 Run the script + capture JSON

```
export BEDROCK_MODEL_ID=apac.anthropic.claude-3-5-sonnet-20241022-v2:0
python3 scripts/generate_architectures.py
# Expected: prints a summary with 2-3 option names + sandbox flags;
# Cascade output: ~35-45 lines; review against Concept 4 template before running.
```

EXPECTED

outputs/m3-arch-options.json has 2-3 options with all required fields.

3 Pair-score on the 5-dim scorecard

```
# Read each option's flow + pros + cons. Pair-discuss for ~5 min per option.
# Score 0-3 per dim. Use Guided Design slide 3's worked example as calibration.
cat > reviews/m3-arch-scorecard.csv <<EOF
```

EXPECTED

/reviews/m3-arch-scorecard.csv has 1 row per dim × N options + totals row + sandbox flag row.

4 Pick the winner + write decision MR

```
# Decision MR shape (1-page; concise):
cat > reviews/m3-arch-decision-MR.md <<'EOF'
# Module 3 — Architecture Decision
```

EXPECTED

/reviews/m3-arch-decision-MR.md captures winner + rationale + runner-up's strongest dim.

5 Verify Bedrock budget + reset prompt env

```
# Track Bedrock usage (the metric you'll watch every module):
PROFILE=apac.anthropic.claude-3-5-sonnet-20241022-v2:0
```

EXPECTED

Bedrock invocations used today logged; 1 required + up to 3-4 if regen; ~46+ left for D4-D12.

6 Commit + push (2 commits)

```
git add scripts/ outputs/ reviews/m3-arch-scorecard.csv reviews/m3-arch-decision-MR.md
git commit -m "module-3: Bedrock arch options + scorecard + decision MR (Lab 2)"
git push
```

EXPECTED

MR shows 2 commits (Lab 1 + Lab 2); pair partner tagged.

Success criteria & common gotchas

✓ YOU ARE READY IF…

· scripts/generate_architectures.py runs cleanly; asserts 2-3 options; FLAGS sandbox_compatible=False without crashing
· outputs/m3-arch-options.json has the JSON response (2-3 options, each with all required fields)
· /reviews/m3-arch-scorecard.csv has totals + sandbox flag; scorecard hard-disqualifies any incompatible options
· /reviews/m3-arch-decision-MR.md captures winner + rationale + runner-up's strongest dim
· Bedrock invocations today: 1 required (Lab 2 script) + up to 3-4 if regenerated
· MR shows 2 commits

⚠ TOP GOTCHAS

Bedrock returned markdown-fenced JSON

Despite SECTION 4 saying 'no markdown fences', sometimes Bedrock wraps JSON in ```json blocks. Use the strip_fences() helper from Concept 4 — handles uppercase fences, trailing prose, leading whitespace. Do NOT rely on raw.removeprefix/removesuffix (brittle for newlines and trailing text).

sandbox_compatible=True but option uses SageMaker

Bedrock self-reports compatibility — verify against your CONSTRAINTS block. If services list includes SageMaker / Bedrock Agents / EventBridge Pipes, the option is NOT compatible regardless of the flag. Hard-disqualify.

Pair-disagreement on a score

Take the LOWER score + note it in the MR. Disagreement is data — captures where the team's review calibration drifts.

Forgot to commit /outputs/ folder

outputs/m3-arch-options.json is the audit artefact; commit it. Without it, Lab 3 (and the trainer) can't reproduce the decision.

Single-option output

Bedrock returned 1 option only — the prompt's TASK section was too narrow. Tighten the prompt to require 'EXACTLY 2-3 options'; re-run.

LAB 3 · ~45 MINUTES

Cascade-critique a deliberately weak function + refine the prompt + re-run

BRIEF

Solo. Trainer provides src/weak_validator/handler.py — a Cascade-generated cancellation-event validator that exhibits all 6 failure modes from Concept 5. Run the 5-dim HITL review on it; score it (it should fail, ~5/15). Identify which failure modes triggered which dim drops. Then write prompts/v2_validator.txt — a tighter prompt that closes the 6 failure modes. Re-run via Cascade; score the new output; show the diff (~14/15). Commit both reviews + the diff.

TIME

45 min

LEVEL

Working+

DELIVERABLE

src/weak_validator/handler.py (trainer-provided; do NOT modify) + /reviews/HITL_REVIEW-weak_validator.md (5-dim score; expected ~5/15; lists which failure modes triggered) + prompts/v2_validator.txt (refined 5-section prompt; closes the 6 failure modes) + src/strong_validator/handler.py (Cascade-generated against v2 prompt) + /reviews/HITL_REVIEW-strong_validator.md (5-dim score; expected ~14/15) + /reviews/m3-prompt-refinement-diff.md (what changed in v2 + why each change closes a failure mode)

Six steps. Pull weak fn → review → identify failure modes → refine prompt → Cascade re-run → diff.

1 Pull the trainer-provided weak validator

```
# PRIMARY PATH (recommended) — trainer provides a known-bad weak validator:
mkdir -p src/weak_validator
cp /trainer/m3/weak_validator.py src/weak_validator/handler.py
# This guarantees the same 6 failure modes for everyone — Cascade may refuse
```

EXPECTED

src/weak_validator/handler.py exists; ~30 lines; visibly bad (do not fix yet).

2 Run the 5-dim HITL review

```
cat > reviews/HITL_REVIEW-weak_validator.md <<'EOF'
# HITL Review — src/weak_validator/handler.py
**Reviewer:** <learner>  **Date:** <YYYY-MM-DD>
#           use boto3.client('s3').get_object_async(); use  order_id  field;
```

EXPECTED

/reviews/HITL_REVIEW-weak_validator.md scored ~5/15 with each dim drop tied to a Concept-5 failure mode.

3 Author prompts/v2_validator.txt — refined to close the 6 failure modes

```
cat > prompts/v2_validator.txt <<'EOF'
## ROLE
  You are a senior AWS data engineer specialising in airline event validation
```

EXPECTED

5-section prompt with explicit constraints addressing each failure mode found.

4 Cascade-author the strong validator against v2 prompt

```
@cascade  Read prompts/v2_validator.txt + .windsurfrules + EXAMPLES.
          Author src/strong_validator/handler.py.
          Include a docstring at top listing which Concept-5 failure mode each
```

EXPECTED

src/strong_validator/handler.py — pure event-validator; no failure modes; correct field names; classified errors.

5 Re-run the 5-dim HITL review on the strong version

```
cat > reviews/HITL_REVIEW-strong_validator.md <<'EOF'
# HITL Review — src/strong_validator/handler.py
**Reviewer:** <learner>  **Date:** <YYYY-MM-DD>
#   - Use powertools Logger
```

EXPECTED

/reviews/HITL_REVIEW-strong_validator.md scored ~14/15 (or higher).

6 Write the prompt-refinement diff + commit

```
cat > reviews/m3-prompt-refinement-diff.md <<'EOF'
# Prompt Refinement Diff — v1 → v2
Weak validator (no .windsurfrules, no v2-style REVIEW_NOTES) scored 5/15
## REVIEW_NOTES (closes 6 failure modes from Concept 5)
## What v2 added (each line maps to a Concept-5 failure mode it closes)
```

EXPECTED

/reviews/m3-prompt-refinement-diff.md explains what v2 added vs v1 + each change to a failure mode.

Success criteria & common gotchas

✓ YOU ARE READY IF…

· src/weak_validator/handler.py committed (do NOT fix it; it's the anti-example)
· /reviews/HITL_REVIEW-weak_validator.md scored ~5/15 with each dim drop tied to a Concept-5 failure mode
· prompts/v2_validator.txt has explicit constraints closing all 6 failure modes
· src/strong_validator/handler.py scored ~14/15 (≥12 pass bar)
· /reviews/m3-prompt-refinement-diff.md explains the prompt changes + which dim improved
· MR shows 3 commits total (Lab 1 + Lab 2 + Lab 3)

⚠ TOP GOTCHAS

Skipping the weak-validator review

If you go straight to the strong version, you don't internalise WHY the failure modes matter. The weak review is the teaching artefact.

Generic REVIEW NOTES

'Be careful' / 'use best practices' don't constrain Cascade. Specific bans + specific requirements (with field names + ARN patterns) do.

Strong validator still scored <12

Either the v2 prompt's REVIEW NOTES were not specific enough, OR you forgot to commit .windsurfrules to repo root. Re-run the test from Lab 1 to verify Cascade is reading it.

Did not include docstring mapping failure modes

v2 prompt asked Cascade to include a docstring listing which Concept-5 failure mode each design choice closes. Without it, the reviewer can't audit the closures cheaply.

Re-using src/weak_validator/ name for the strong version

Keep both. weak_validator stays as the anti-example for the cohort retrospective; strong_validator is the reference for D4.

Code shipped. Branches pushed. Time to step back.

04

PART 4 · ~20-30 MINUTES

Review & wrap

Five Day-3 anti-patterns to avoid. End-of-module cleanup (light — no AWS provisioning today). Day-3 in numbers (0 new Lambdas; ~1-4 Bedrock invocations (Lab 2 only; Cascade IDE not counted); 4 audit folders + .windsurfrules v1 + 2 reviews + 1 decision MR). Bridge to Day 4 — Batch Pipeline Design on AWS, where today's prompt + .windsurfrules drive the design doc.

Five Day-3 anti-patterns — and what to do instead

01 Treating Cascade like autocomplete

MISTAKE Accepting Cascade output without reading it. Hitting Tab on multi-file scaffolds. Skipping the 5-dim HITL review.

DO INSTEAD Agentic mode = 5-10 min review per output, not 1 second. The 5-dim checklist is the gate; without it Cascade ships hallucinated APIs into production.

02 Vague REVIEW NOTES section

MISTAKE 'Be careful with security' / 'use best practices' / 'follow conventions'. Cascade can't act on these; output stays generic.

DO INSTEAD REVIEW NOTES are explicit bans + explicit requirements. 'NO Action:* in IAM'; 'use field name pnr NOT order_id'; 'classify ClientError + SchemaError separately'. Every D4-D12 prompt follows this pattern.

03 Skipping the weak-vs-strong contrast

MISTAKE Going straight to the strong validator without reviewing the weak one. Failure modes stay abstract; team forgets what to scan for.

DO INSTEAD Lab 3's weak-then-strong is the teaching artefact. The HITL review of the weak version is what makes the failure modes concrete + memorable.

04 Uncommitted prompts

MISTAKE Cascade output is committed; the prompt that produced it is not. Reviewer can't tell whether code is right but prompt was vague, or prompt was tight but Cascade hallucinated.

DO INSTEAD EVERY prompt goes in prompts/v<N>_<purpose>.txt + git commit. .windsurfrules REPO RULES makes this mandatory; D4-D12 reviewers refer to the prompt in HITL_REVIEW.md.

05 Not testing .windsurfrules pickup

MISTAKE Saved .windsurfrules at repo root + assumed Cascade picked it up. First D4 lab fails because the file was at wrong path or chat was opened before save.

DO INSTEAD Lab 1's test prompt — 'scaffold a hello-world Lambda for the cancellation envelope' — must produce output using src/<component>/ + airline fields. If not, .windsurfrules is not being read; fix path or open new chat.

End-of-module cleanup (light — no AWS provisioning today)

CLEANUP CHECKLIST

· Branch pushed: module-3-<learner> with 3 commits (Lab 1 + Lab 2 + Lab 3)
· MR open; pair partner tagged
· .windsurfrules at repo root with 4 sections (REPO RULES + CONSTRAINTS + EXAMPLES + REVIEW)
· /prompts/, /reviews/, /design/ folders committed (will be filled D4-D12)
· Bedrock invocations today: ~1-4 of 50 (Lab 2 = 1 required + up to 3 if regen; Lab 3 Cascade IDE usage does NOT count against AWS Bedrock Runtime — only direct boto3.bedrock-runtime calls do)
· 0 new Lambdas today — cumulative still 1 (D2 starter validator). 9 left for D4-D12.
· 0 new AWS resources today (no provisioning) — sandbox in same state as end of D2
· src/weak_validator/handler.py + src/strong_validator/handler.py preserved as references for D4

```
# Verify nothing was provisioned by accident
aws lambda list-functions --region ap-south-1 \
  --query "length(Functions[?contains(FunctionName,'$LEARNER')])"
# Expected: 1 (the D2 starter validator) — unchanged

# Bedrock budget check (the metric you'll watch every module):
PROFILE=apac.anthropic.claude-3-5-sonnet-20241022-v2:0

# Cross-platform UTC timestamps (works on macOS + Linux):
START=$(python3 -c 'from datetime import datetime,timezone; \

print(datetime.now(timezone.utc).replace(hour=0,minute=0,second=0,microsecond=0).is
oformat().replace("+00:00","Z"))')
END=$(python3 -c 'from datetime import datetime,timezone; \
  print(datetime.now(timezone.utc).isoformat().replace("+00:00","Z"))')

for METRIC in Invocations InputTokenCount OutputTokenCount; do
  aws cloudwatch get-metric-statistics --namespace AWS/Bedrock \
    --metric-name $METRIC --dimensions Name=ModelId,Value=$PROFILE \
    --start-time $START --end-time $END --period 3600 --statistics Sum \
    --region ap-south-1
done
# Expected: Invocations ~1-4 today (cumulative ~2-5 of 50 including Day 1 verify).
```

Trainer note: Bedrock budget healthy (~2-5 of 50 used; 45+ left). Lambda count unchanged at 1 of 10. Day 4 (Batch Pipeline Design) reuses today's .windsurfrules + 5-section prompt; Day 5 promotes .windsurfrules to v2 by adding a BATCH CONTEXT block (Records[], partial-batch protocol, idempotency contract). Today's audit folders are the input to D4's design doc.

What you actually shipped

0

NEW LAMBDAS

Foundation day. No AWS provisioning. Cumulative still 1 of 10 (D2 starter).

1

.WINDSURFRULES v1

4 sections committed. Cascade should pick up on every prompt (Lab 1 verifies) D4-D12. Each module extends with 1 new context layer.

5 → 14/15

WEAK → STRONG

Lab 3 weak validator scored 5/15 across 6 failure modes. v2 prompt closed them; strong scored 14/15.

~10/50

BEDROCK USED

Lab 2 architecture options + Lab 3 Cascade refinements. ~40 left for D4-D12.

DAY 3 → DAY 4 · WHAT'S COMING TOMORROW

Day 4 — Batch Pipeline Design on AWS

Today set up the prompt + context discipline. Tomorrow the first real engineering module: design the four-zone airline batch pipeline (raw → validated → curated → quarantine) on paper before any code lands. Failure modes catalogued. Idempotency keys chosen. .windsurfrules gets a new section: 8th BATCH CONTEXT layer (Records[], partial-batch protocol, idempotency contract).

DESIGN BEFORE CODE

0 new Lambdas tomorrow either — design only

D4 produces /design/m4-batch-pipeline-design.md + a failure-mode catalogue + idempotency table schema. D5 builds the validator + chunker + lander + dlq_replayer Lambdas (4 new) against tomorrow's design.

8TH PROMPT LAYER

BATCH CONTEXT (Records[], partial-batch protocol, idempotency)

.windsurfrules v2 adds the BATCH CONTEXT section between CONSTRAINTS and EXAMPLES. Cascade now scaffolds batch handlers with batchItemFailures + claim-and-complete + classified errors.

SANDBOX BUDGET

0 new Lambdas + ~5 Bedrock invocations (design exercise)

Cumulative after Day 4: still 1 of 10 Lambdas; ~15 of 50 Bedrock invocations. D5 introduces the first batch Lambdas (4 new → 5 of 10 cumulative).

Day 4 branch: module-4-<learner>. Inherits today's .windsurfrules v1 + the 5-dim HITL review template. Bring your prompt-authoring discipline — D4's design doc lives or dies on the prompt that produces it.

DAY 3 · WRAPPED

AI is now a discipline, not autocomplete.

.WINDSURFRULES v1

4 sections committed. Cascade should pick up on every prompt (Lab 1 verifies) D4-D12. Extends with 1 new context layer per module.

5-DIM HITL CHECKLIST

Correctness · Security · Observability · Scalability · Cost. Pass: >=12/15 AND every dim >=2/3. Used on every artefact D4-D12.

AUDIT TRAIL

/prompts/ + /reviews/ + /design/ folders committed. Every Cascade output traceable to the prompt that produced it.

Day 4 starts at 09:00. Bring your .windsurfrules + a clear head — no AWS provisioning tomorrow either; we design the four-zone batch pipeline before any code lands.

Accelya Kale · Agentic Development using AWS · Trainer: Jai Singh